

PilotFish HL7 LLP Listener – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

Think of this as an **HL7 mailbox on a network port**. External systems connect over TCP using the MLLP framing protocol (start byte, message, end byte). Each complete HL7 message starts an integration transaction; optionally the listener waits for your route to finish and sends back an ACK over the same connection.

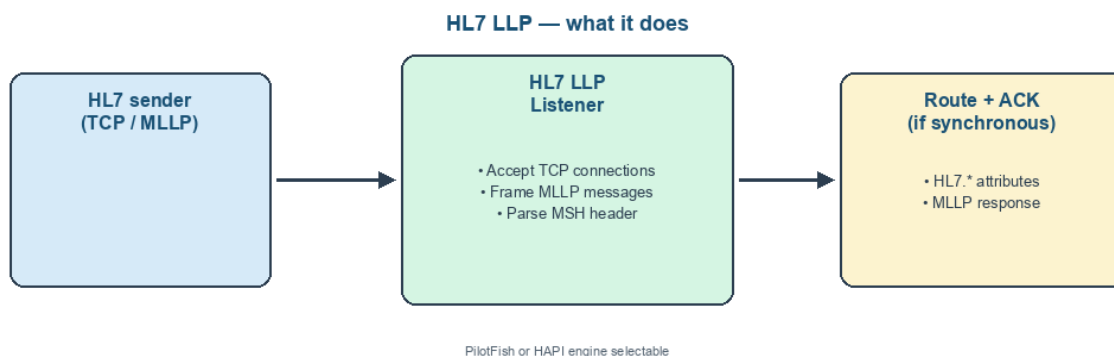


Figure 1: At a glance (plain English)

- **Listens** on a TCP port for HL7 v2 messages wrapped in MLLP.
- **Extracts** MSH header fields into named transaction attributes.
- **Supports** PilotFish-native or HAPI-based parsing engines.
- **Can reply** synchronously with custom or auto-generated ACKs.

Purpose

Deep dive into the **HL7 LLP** listener in **eip.war.hs.26R1.11**: engine selection, MLLP state machine, synchronous ACK flow, compliance options, and configuration.

Provenance	Value
WAR	<code>eip.war.hs.26R1.11</code>
Module JAR	<code>xcs-format-hl7-1.0.1.jar</code>
Core JAR	<code>xcs-core-1.2.6.jar</code>
Decompiler	CFR 0.152
Implementation class	<code>com.pilotfish.eip.modules.hl7.HL7TCPListen</code>
Decompiled path	<code>war-pipeline/decompiled/eip-26R1.11/.../HL</code>
Listener type string	HL7 LLP
Description	Monitors a specified TCP socket for HL7 messages
Help ID	<code>console.listener.hl7</code>

Related classes:

Class	Role
<code>HL7TCPLListener</code>	Configuration, attribute registration, engine dispatch
<code>HL7SimpleListener</code>	PilotFish MLLP parser + ACK builder
<code>HL7HapiListener</code>	HAPI-based engine with validation/compliance
<code>AbstractListener</code>	Synchronous dispatch, throttling, base options

1. High-level behavior

1. **startListening** chooses engine from **HL7 Engine** (PilotFish → HL7SimpleListener, HAPI → HL7HapiListener), then calls `super.startListening()`.
2. **engageListener** delegates to the engine implementation (binds `ServerSocket` on **Port** for PilotFish engine).
3. **Accept loop** (PilotFish): fixed thread pool `1 + MaximumConnections`; each client socket processed on worker thread.
4. **MLLP reader** accumulates message into cache until end frame; parses MSH fields into attributes.
5. **Dispatch**: async `dispatchListeners` or sync `dispatchForSynchronousResponseTransactionData` with **Synchronous Timeout**.
6. **Response path**: route output stream and/or auto-generated ACK sent as MLLP-framed bytes on the socket.

Client TCP connect

-> MLLP state machine (0x0B ... 0x1C 0x0D)

```

-> cache full message
-> set HL7.MSH* attributes
[Synchronous?]
  yes -> wait route; build/send ACK (auto or stream)
  no  -> dispatch; optional auto-ACK if enabled

```

2. Configuration reference

2.1 Basic

Label	Tag	Default	Notes
HL7 Engine	Engine	PilotFish (UI); legacy XML missing tag → HAPI	PilotFish = granular ACK control
Port	Port	-1 (required)	Server socket port; range 1–65536
Socket Timeout	TimeoutTag	0	SO_TIMEOUT seconds; 0 = no read timeout
Synchronous	SynchronousTag	false	Wait for route response before ACK
Synchronous Timeout	SynchronousTimeout	30	Visible when Synchronous true
Auto-Generate ACK	AutogenerateSynchronousACK	false	Build ACK from response attributes
Maximum Connections	MaximumConnections	5	PilotFish engine only

2.2 HAPI engine options

Visible when Engine = **HAPI**:

Label	Tag	Default
Validate Messages	Validate	false
2.7 Compliance Mode	EnableComplianceMode	false
Invalid Character Handling	InvalidCharacterHandling	Reject
Disconnect on Error	CloseOnError	false

Compliance dependencies: when 2.7 Compliance Mode true, shows Invalid Character Handling and Disconnect on Error.

Invalid Character Handling choices: **Reject**, **Ignore**, **Replace**.

2.3 Legacy configuration migration

readConfiguration: if route XML has **no Engine** element, value forced to **HAPI** on load (backward compatibility for older routes).

3. MLLP protocol (PilotFish engine)

Framing bytes:

Byte	Value	Role
Start	0x0B (VT)	Begin message
Segment end	0x0D (CR)	HL7 segment terminator
End	0x1C (FS) + 0x0D	MLLP message end

State machine reads segment names (3 chars), special-cases header segments (**FHS**, **BHS**, **MSH**), rejects control chars $\leq 0x1F$ except CR in body.

On parse error, sends **AE** MSA NACK with error text. On success (async + auto-ACK), sends **AA** MSA ACK with control ID from inbound MSH.

4. Transaction attributes

Registered in `registerTransactionAttributes` (design-time metadata):

Attribute	Meaning
<code>HL7.SynchronousResponse.Code</code>	Success/Failure for ACK generation
<code>HL7.SynchronousResponse.Message</code>	ACK message text
<code>HL7.FieldSeparator</code>	MSH-1
<code>HL7.EncodingCharacters</code>	MSH-2
<code>HL7.SendingApplication</code>	MSH-3
<code>HL7.SendingFacility</code>	MSH-4
<code>HL7.ReceivingApplication</code>	MSH-5
<code>HL7.ReceivingFacility</code>	MSH-6
<code>HL7.DateTimeOfMessage</code>	MSH-7
<code>HL7.Security</code>	MSH-8
<code>HL7.MessageType</code>	MSH-9
<code>HL7.MessageControlID</code>	MSH-10
<code>HL7.ProcessingID</code>	MSH-11
<code>HL7.VersionID</code>	MSH-12

Also registered (HTTP-style, reused naming): `com.pilotfish.RequestPath`, `ServerName`, `ServerPort`, `RemoteAddr`, `RemoteHost`, `RequestURL`, `Protocol`, `Locale`.

Response code constants: `Success`, `Failure` (`RESPONSE_CODE_SUCCESS` / `RESPONSE_CODE_FAILURE`).

5. Synchronous ACK modes

When **Synchronous** = true:

1. Route runs with **Synchronous Timeout** seconds.
2. If **Auto-Generate ACK** = true:
 - Reads `HL7.SynchronousResponse.Code` from response attributes.
 - **Success** → AA MSA; **Failure** → AE MSA with message.
3. If **Auto-Generate ACK** = false, transaction **output stream** is treated as the acknowledgement payload (MLLP-framed by caller/route).

When **Synchronous** = false and **Auto-Generate ACK** = true, AA ACK sent immediately after dispatch (PilotFish engine path).

6. Module capabilities

Capability	ID	Description
connectionStatus	connectionStatus	Lists current client sockets and connected flag

Available for both `HL7SimpleListener` and `HL7HapiListener` via `executeCapability`.

7. Known quirks

1. **Default engine mismatch:** UI default PilotFish, but legacy routes without **Engine** element load as **HAPI**.
2. **Maximum Connections** only applies to PilotFish engine UI dependency.
3. **Socket timeout** closes connection on read timeout (logged at DEBUG).
4. **Exceeding max connections** throws error on accept path.
5. **Synchronous timeout** returns internal AE NACK (“Internal timeout”) on PilotFish engine.
6. **MSH date format** helper uses pattern `yyyyMMddhhmmss` (12-hour `hh` in `SimpleDateFormat`).
7. **HL7TCPLListener.doListening** delegates to super — primary work is in engine `engageListener`, not polling loop.

8. Operational recommendations

Goal	Settings
Full ACK control in route	Engine = PilotFish; Synchronous + manual ACK payload
Strict HL7 validation	Engine = HAPI; Validate Messages = true

Goal	Settings
High connection volume	Increase Maximum Connections; monitor via <code>connectionStatus</code>
Long-running routes	Increase Synchronous Timeout
Legacy route audit	Explicitly set Engine after upgrade; don't rely on XML default
Idle connections	Set Socket Timeout > 0 to reap stale clients

9. Worked example (synchronous ADT)

Configuration: Engine PilotFish, Port 2575, Synchronous true, Synchronous Timeout 60, Auto-Generate ACK true.

1. Sending system connects, transmits `ADT^A01` in MLLP frame.
2. Listener parses MSH, sets attributes, dispatches synchronously.
3. Route sets `HL7.SynchronousResponse.Code = Success`.
4. Listener builds `MSH + MSA|AA|{control ID}` frame and returns on socket.

If route sets `Code = Failure` with message, listener sends `MSA|AE` instead.

10. Inherited AbstractListener options

Standard base: trigger-only init, CLI, restart on error, FIFO, throttling, inactivity, scheduling. HL7 listener uses synchronous dispatch helpers from `AbstractListener` for sync mode.

11. Source index

Topic	Path
HL7TCPLListener	<code>com/pilotfish/eip/modules/hl7/HL7TCPLListener.java</code>
HL7SimpleListener	<code>com/pilotfish/eip/modules/hl7/HL7SimpleListener.java</code>
HL7HapiListener	<code>com/pilotfish/eip/modules/hl7/HL7HapiListener.java</code>

Deep-dive documentation from WAR decompile — 26R1.11 — August 2026.